Preparation Notebook

Setup Environment

```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.9/12.9 MB 40.8 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.9/4.9 MB 50.9 MB/s eta
0:00:00
ERROR: pip's dependency resolver does not currently take into account
all the packages that are installed. This behaviour is the source of
the following dependency conflicts.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-
learn 1.5.2 which is incompatible.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn
1.5.2 which is incompatible.
Mounted at /content/gdrive

You can now save your data files in:
/content/gdrive/MyDrive/36106/assignment/AT3/data
```

Student Information

```
# <Student to fill this section>
group_name = "Group 20"
student_name = "Ratnadeep Patra"
student_id = "26294153"

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='group_name', value=group_name)
```

```
<IPython.core.display.HTML object>
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_name', value=student_name)
```

```
<IPython.core.display.HTML object>
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_id', value=student_id)
```

```
<IPython.core.display.HTML object>
```

---

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

```
# <Student to fill this section and then remove this comment>
```

0.b Import Packages

```
# DO NOT MODIFY THE CODE IN THIS CELL
import pandas as pd
import altair as alt

import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from collections import defaultdict
import graphviz
from sklearn.preprocessing import StandardScaler
import pickle

pd.set_option('display.max_columns', None) # Forcing pandas to display
all columns
pd.set_option('display.max_rows', None) # Forcing pandas to display
all rows
```

A. Feature Selection

A.0 Load Data

```
# DO NOT MODIFY THE CODE IN THIS CELL
# Load datasets
```

```
try:
  customer_df = pd.read_csv(at.folder_path / "customer.csv")
  person_df = pd.read_csv(at.folder_path / "person.csv")
  product_category_df = pd.read_csv(at.folder_path /
"product_category.csv")
  product_cost_history_df = pd.read_csv(at.folder_path /
"product_cost_history.csv")
  product_list_price_history_df = pd.read_csv(at.folder_path /
"product_list_price_history.csv")
  product_sub_category_df = pd.read_csv(at.folder_path /
"product_sub_category.csv")
  product_df = pd.read_csv(at.folder_path / "product.csv")
  sales_order_detail_df = pd.read_csv(at.folder_path /
"sales_order_detail.csv")
  sales_order_header_df = pd.read_csv(at.folder_path /
"sales_order_header.csv")
  sales_territory_df = pd.read_csv(at.folder_path /
"sales_territory.csv")
  special_offer_product_df = pd.read_csv(at.folder_path /
"special_offer_product.csv")
  special_offer_df = pd.read_csv(at.folder_path / "special_offer.csv")
  store_df = pd.read_csv(at.folder_path / "store.csv")
  unit_measure_df = pd.read_csv(at.folder_path / "unit_measure.csv")
except Exception as e:
  print(e)
```

A.1 Approach 1 "Merge required df"

```
df_dict = {
    "customer": customer_df,
    "person": person_df,
    "product_category": product_category_df,
    "product_cost_history": product_cost_history_df,
    "product_list_price_history": product_list_price_history_df,
    "product_sub_category": product_sub_category_df,
    "product": product_df,
    "sales_order_detail": sales_order_detail_df,
    "sales_order_header": sales_order_header_df,
    "sales_territory": sales_territory_df,
    "special_offer_product": special_offer_product_df,
    "special_offer": special_offer_df,
    "store": store_df,
    "unit_measure": unit_measure_df
}

pk_df_map = {
    "customer_id" : "customer",
    "person_id" : "person",
    "product_category_id" : "product_category",
    "product_subcategory_id" : "product_sub_category",
```

```
    "product_id" : "product",
    "sales_order_detail_id" : "sales_order_detail",
    "sales_order_id" : "sales_order_header",
    "sales_territory_id" : "sales_territory",
    "special_offer_id" : "special_offer",
    "store_id" : "store",
    "unit_measure_code" : "unit_measure",
    "size_unit_measure_code" : "unit_measure",
    "weight_unit_measure_code" : "unit_measure",
    "territory_id" : "sales_territory"
} # Identified from other EDA files

column_map = defaultdict(list)
for table_name, df in df_dict.items():
    for col in df.columns:
        column_map[col].append(table_name)

fk_relations = []
for col, tables in column_map.items():
    parent_table = None
    for t in tables:
        if col.lower() in pk_df_map.keys():
            parent_table = pk_df_map[col.lower()]
            break
    if parent_table is not None:
        for child_table in tables:
            if child_table != parent_table:
                fk_relations.append({
                    "child_table": child_table,
                    "parent_table": parent_table,
                    "fk_column": col
                })

dot = graphviz.Digraph(
    comment='Entity Relationship Diagram',
    graph_attr={
        'rankdir': 'LR',
        'size': '10,6',
        'ratio': 'compress'})
for table_name, df in df_dict.items():
    pk_cols_for_table = {pk_col_name for pk_col_name, pk_table_name in
pk_df_map.items() if pk_table_name == table_name}
    fk_cols_for_table = {rel['fk_column'] for rel in fk_relations if
rel['child_table'] == table_name}
    current_table_pks = []
    current_table_fks = []
    current_table_others = []

    for col in df.columns.tolist():
        if col in pk_cols_for_table:
```

```
            current_table_pks.append(f'<FONT
COLOR="red">{col}</FONT>')
        elif col in fk_cols_for_table:
            current_table_fks.append(f'<FONT
COLOR="blue">{col}</FONT>')
        else:
            current_table_others.append(col)

    column_labels_formatted = current_table_pks + current_table_fks +
current_table_others
    columns_html = '<br/>'.join(column_labels_formatted)
    label = f"<<B>{table_name}</B><br/>{columns_html}>"
    dot.node(table_name, label=label, shape='box')

for rel in fk_relations:
    dot.edge(rel['parent_table'],
             rel['child_table'],
             label=rel['fk_column'],
             style="dashed")

display(dot)
```

```
merged_sales_df = pd.merge(sales_order_detail_df,
sales_order_header_df, on='sales_order_id', how='left')
merged_sales_df = pd.merge(merged_sales_df, special_offer_df,
on='special_offer_id', how='left')
product_df.drop_duplicates(inplace=True) # As identified during EDA,
product table has a lot of duplicates
merged_sales_df = pd.merge(merged_sales_df, product_df,
on='product_id', how='left', suffixes=('', '_product'))
merged_sales_df = pd.merge(merged_sales_df, product_sub_category_df,
on='product_subcategory_id', how='left', suffixes=('',
'_subcategory'))
merged_sales_df = pd.merge(merged_sales_df, product_category_df,
on='product_category_id', how='left', suffixes=('', '_category'))

len(merged_sales_df) == len(sales_order_detail_df)
```

```
True
```

```
# <Student to fill this section>
feature_selection_1_insights = """
The initial feature selection focused on merging the core tables
required to study product-level sales behaviour and identify unusual
sales patterns for
products (as per the buisness use case stated in Anomaly detection
notebook). Peripheral tables such as 'person', 'store', and
'sales_territory' were
excluded to maintain focus on the immediate transactional and product-
related attributes. History tables such as 'product_cost_history' and
'product_list_price_history' were also omitted to simplify the
analysis and focus on the current sales structure. To handle
overlapping column names across
joined tables, informative suffixes such as '_product' and '_category'
were applied. This produced a flat analytical dataset suitable for
inspecting
transaction-level and product-level features.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_1_insights',
value=feature_selection_1_insights)
```

```
<IPython.core.display.HTML object>
```

A.2 Approach 2 "Drop ID columns"

```
feature_sel_df = merged_sales_df.copy()

id_cols = ['sales_order_id',
 'sales_order_detail_id',
 'product_id',
 'special_offer_id',
```

```
 'sales_person_id',
 'territory_id',
 'currency_rate_id',
 'product_category_id',
 'product_subcategory_id',
 'product_model_id',
 'sales_order_number',
 'customer_id',
 'account_number'
]

feature_sel_df.drop(id_cols, axis=1, inplace=True)

# <Student to fill this section>
feature_selection_2_insights = """
Identification columns such as 'sales_order_id', 'product_id', and
'customer_id' provide unique keys for database integrity but do not
contain any signal
for statistical analysis. Keeping these features would lead to high-
cardinality noise and potentially cause the model to focus on specific
entity instances
rather than looking for generalizable patterns. 'account_number' is
also considered an ID as it uniquely identifies a customer's billing
account. By
removing these 13 ID-related columns, we reduce the dimensionality of
the dataset and ensure the model focuses on meaningful attributes like
price, quantity,
and product characteristics. Although 'customer_id' and
'account_number' are dropped currently, later, once an anomaly is
detected, they can be again used
to identify the reason for that anomaly and further analysis around
that can be done to check if the anomaly is specific to a customer or
some specific
account number of a customer to get the root cause of that anomaly.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_2_insights',
value=feature_selection_2_insights)
```

```
<IPython.core.display.HTML object>
```

A.3 Approach 3 "Drop constant value features"

```
feature_sel_df['is_sellable'].value_counts()
```

```
is_sellable
1.0    42100
Name: count, dtype: int64
```

```
constant_value_col = [
    'status',
    'is_sellable',
    'discontinued_date'
]

feature_sel_df.drop(constant_value_col, axis=1, inplace=True)

# <Student to fill this section>
feature_selection_3_insights = """
The features 'status', 'is_sellable', and 'discontinued_date' were
removed due to their lack of variance, making them unsuitable for any
ML algorithm.
During the EDA of the 'sales_order_header' table, it was observed that
all samples contained a constant value of 5 for this column. A feature
with a
single unique value provides no significant signals for any model.
Similarly, for the 'is_sellable' column, each sample is having value 1
as all products
recorded in the sales data should be eligible to sell, rendering this
feature constant and uninformative for predicting anomalies. The EDA
of the 'product'
table revealed that the 'discontinued_date' column contained
exclusively NaN values. A feature composed entirely of missing values
carries no information.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_3_insights',
value=feature_selection_3_insights)
```

```
<IPython.core.display.HTML object>
```

A.4 Approach 4 "Drop derived features"

```
feature_sel_df[
    feature_sel_df['total_due'].round(4) != (
        feature_sel_df['sub_total']
        + feature_sel_df['tax_amount']
        + feature_sel_df['freight']
    ).round(4)
][['sub_total', 'tax_amount', 'freight', 'total_due']]
```


```
{"summary":"{\n  \"name\": \"[['sub_total', 'tax_amount', 'freight',
'total_due']]\",\n  \"rows\": 0,\n  \"fields\": [\n    {\n
\"column\": \"sub_total\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": null,\n        \"min\":
null,\n        \"max\": null,\n        \"num_unique_values\": 0,\n
\"samples\": [],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
```

\"tax_amount\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": null,\n   \"min\": null,\n
\"max\": null,\n   \"num_unique_values\": 0,\n
\"samples\": [],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"freight\",\n   \"properties\": {\n   \"dtype\": \"number\",\
n   \"std\": null,\n   \"min\": null,\n   \"max\":
null,\n   \"num_unique_values\": 0,\n   \"samples\": [],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"total_due\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
null,\n   \"min\": null,\n   \"max\": null,\n
\"num_unique_values\": 0,\n   \"samples\": [],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe"}

```
feature_sel_df['sales_order_id'] = merged_sales_df['sales_order_id']
sales_order_summary = feature_sel_df.groupby('sales_order_id').agg(
    sum_line_total=('line_total', 'sum'),
    sub_total=('sub_total', 'first')
).reset_index()
feature_sel_df.drop(['sales_order_id'], axis=1, inplace=True)
sales_order_summary[
    sales_order_summary['sum_line_total'].round(4) !=
sales_order_summary['sub_total'].round(4)
]
```

{"summary":"{\n \"name\": \"]\",\n \"rows\": 9,\n \"fields\": [\n
{\n   \"column\": \"sales_order_id\",\n   \"properties\": {\n
\"dtype\": \"string\",\n   \"num_unique_values\": 9,\n
\"samples\": [\n   \"b46c1a2f-fab3-45d6-838c-dca0a55b3a75\",\n
\"11bea793-bc5e-4af3-a3aa-1516829af0ae\",\n   \"b209b701-50e8-
4d9f-b821-9c26b7300c92\"\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \"sum_line_total\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 19645.15073410285,\n
\"min\": 8617.549649999999,\n   \"max\": 72301.95105,\n
\"num_unique_values\": 9,\n   \"samples\": [\n
29331.50985,\n   38818.22895,\n   8617.549649999999\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"sub_total\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
19645.15073410285,\n   \"min\": 8617.5497,\n   \"max\":
72301.9511,\n   \"num_unique_values\": 9,\n   \"samples\":
[\n   29331.5099,\n   38818.229,\n   8617.5497\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   }\n]\n}","type":"dataframe"}

```
sales_order_summary[
    ~np.isclose(sales_order_summary['sum_line_total'],
```

```
sales_order_summary['sub_total'].round(4))
]
```


```
{"repr_error":"Out of range float values are not JSON compliant:
nan","type":"dataframe"}
```


```
feature_sel_df[
    feature_sel_df['line_total'].round(4) != (
        feature_sel_df['order_quantity'] *
feature_sel_df['unit_price'] * (1 -
feature_sel_df['unit_price_discount'])
    ).round(4)
][['line_total', 'order_quantity', 'unit_price',
'unit_price_discount']]
```


```
{"summary":"{\n  \"name\": \"[['line_total', 'order_quantity',
'unit_price', 'unit_price_discount']]\",\n  \"rows\": 8,\n
\"fields\": [\n    {\n      \"column\": \"line_total\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
2519.3180454163466,\n        \"min\": 46.93095,\n        \"max\":
7364.02095,\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          46.93095,\n          84.55095,\n          7364.02095\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"order_quantity\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.5175491695067657,\n        \"min\": 17.0,\n        \"max\": 18.0,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          17.0,\n
18.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"unit_price\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 147.13506673921307,\n        \"min\":
2.7445,\n        \"max\": 430.6445,\n        \"num_unique_values\":
4,\n        \"samples\": [\n          2.7445,\n          4.9445\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"unit_price_discount\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
7.417989609027187e-18,\n        \"min\": 0.05,\n        \"max\":
0.05,\n        \"num_unique_values\": 1,\n        \"samples\": [\n
0.05\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


```
feature_sel_df[
    ~np.isclose(feature_sel_df['line_total'], (
        feature_sel_df['order_quantity'] *
feature_sel_df['unit_price'] * (1 -
feature_sel_df['unit_price_discount'])
    ))
][['line_total', 'order_quantity', 'unit_price',
'unit_price_discount']]
```

{"summary":"{\n \"name\": \"[['line_total', 'order_quantity', 'unit_price', 'unit_price_discount']]\",\n \"rows\": 0,\n \"fields\": [\n   {\n   \"column\": \"line_total\",\n \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": null,\n   \"min\": null,\n   \"max\": null,\n \"num_unique_values\": 0,\n   \"samples\": [],\n \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"order_quantity\",\n \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": null,\n   \"min\": null,\n   \"max\": null,\n \"num_unique_values\": 0,\n   \"samples\": [],\n \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"unit_price\",\n \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": null,\n   \"min\": null,\n   \"max\": null,\n \"num_unique_values\": 0,\n   \"samples\": [],\n \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"unit_price_discount\",\n \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": null,\n   \"min\": null,\n   \"max\": null,\n \"num_unique_values\": 0,\n   \"samples\": [],\n \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```
# total_due = sub_total + tax_amount + freight
# sub_total = sum(line_total)
# line_total = unit_price * order_quantity * (1 - unit_price_discount)

derived_features = [
    'total_due',
    'sub_total',
    'line_total'
]

feature_sel_df.drop(derived_features, axis=1, inplace=True)

# <Student to fill this section>
feature_selection_4_insights = """
Features such as 'total_due', 'sub_total', and 'line_total' are direct
linear combinations of other existing features within the dataset.
Since these
features can be precisely re-derived from more granular attributes,
retaining them would introduce multicollinearity and redundancy
without adding new
predictive information. By dropping these derived features, the
dimensionality of the dataset is reduced, leading to a simpler and
more efficient model
without any loss of crucial information. This approach ensures that
the model focuses on the fundamental, independent variables.
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_4_insights',
value=feature_selection_4_insights)
```

```
<IPython.core.display.HTML object>
```

A.5 Approach 5 "Drop duplicate features"

```
feature_sel_df[feature_sel_df['discount_pct'] !=
feature_sel_df['unit_price_discount']][
    ['discount_pct', 'unit_price_discount', 'description',
'order_quantity', 'min_quantity']].sample(25, random_state=42)
```


```
{"summary":"{\n  \"name\": \"    ['discount_pct',
'unit_price_discount', 'description', 'order_quantity',
'min_quantity']]\",\n  \"rows\": 25,\n  \"fields\": [\n    {\n
\"column\": \"discount_pct\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 3.5409894674753084e-18,\n
\"min\": 0.02,\n        \"max\": 0.02,\n        \"num_unique_values\":
1,\n        \"samples\": [\n          0.02\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"unit_price_discount\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"description\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 1,\n        \"samples\": [\n          \"Volume
Discount 11 to 14\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"order_quantity\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.0,\n        \"min\": 1.0,\n
\"max\": 1.0,\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          1.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"min_quantity\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.0,\n        \"min\": 11.0,\n
\"max\": 11.0,\n        \"num_unique_values\": 1,\n
\"samples\": [\n          11.0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}
```


```
feature_sel_df[feature_sel_df['discount_pct'] !=
feature_sel_df['unit_price_discount']]['min_quantity'].unique()
```

```
array([11.,  0.])
```

```
feature_sel_df[(feature_sel_df['discount_pct'] !=
feature_sel_df['unit_price_discount']) &
```

```
(feature_sel_df['min_quantity'] == 0)][
    ['discount_pct', 'unit_price_discount', 'description',
'order_quantity', 'min_quantity']]
```


```
{"summary":"{\n  \"name\": \"    ['discount_pct',
'unit_price_discount', 'description', 'order_quantity',
'min_quantity']]\",\n  \"rows\": 16,\n  \"fields\": [\n    {\n
\"column\": \"discount_pct\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.0223606797749979,\n
\"min\": 0.15,\n        \"max\": 0.2,\n        \"num_unique_values\":
2,\n        \"samples\": [\n          0.15,\n          0.2\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"unit_price_discount\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.0,\n        \"min\": 0.0,\n
\"max\": 0.0,\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          0.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"description\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          \"Touring-3000 Promotion\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"order_quantity\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0,\n        \"min\": 1.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          1.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"min_quantity\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe"}
```


```
feature_sel_df[feature_sel_df['description'].isin(['Touring-1000
Promotion',
       'Touring-3000 Promotion'])]['order_quantity'].unique()
```

```
array([ 2.,  1.,  3.,  4.,  8., 15.,  9.,  7., 12.,  6.,  5., 10.,
13.,
       21., 11., 14., 16.])
```

```
feature_sel_df[feature_sel_df['discount_pct'] !=
feature_sel_df['unit_price_discount']]['order_quantity'].unique()
```

```
array([1.])
```

The 'unit_price_discount' represents the actual discount applied during a sale, directly impacting the final price. In contrast, 'discount_pct' indicates a discount based on certain regulations or promotions. All relevant discounts are already reflected in 'unit_price_discount'. Therefore,

'discount_pct' introduces a redundancy of information, as its details are implicitly captured by 'unit_price_discount' and other related features. For a more concise and efficient model, 'discount_pct' can be safely dropped without losing critical information.

```
feature_sel_df[feature_sel_df['unit_price'] !=
feature_sel_df['list_price']][[
    'unit_price', 'list_price', 'order_quantity',
'unit_price_discount', 'max_quantity', 'min_quantity', 'description',
'discount_pct'
]].sample(25)
```


```
{"summary":"{\n  \"name\": \"]]\",\n  \"rows\": 25,\n  \"fields\": [\n
{\n      \"column\": \"unit_price\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 633.7978141227794,\n
\"min\": 14.1289,\n        \"max\": 2181.5625,\n
\"num_unique_values\": 20,\n        \"samples\": [\n          158.43,\
n          323.994,\n          48.594\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"list_price\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
833.5023915818517,\n        \"min\": 24.49,\n        \"max\":
2443.35,\n        \"num_unique_values\": 20,\n        \"samples\": [\n
264.05,\n          539.99,\n          80.99\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"order_quantity\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3.1235663378047005,\n        \"min\": 1.0,\n        \"max\": 13.0,\n
\"num_unique_values\": 9,\n        \"samples\": [\n          10.0,\n
13.0,\n          4.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"unit_price_discount\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.003999999999999999,\n        \"min\":
0.0,\n        \"max\": 0.02,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0.02,\n          0.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"max_quantity\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 14.0,\n        \"max\": 14.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          14.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"min_quantity\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
2.2000000000000006,\n        \"min\": 0.0,\n        \"max\": 11.0,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          11.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"description\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"Volume
Discount 11 to 14\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
```

\"discount_pct\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.003999999999999999,\n   \"min\":
0.0,\n   \"max\": 0.02,\n   \"num_unique_values\": 2,\n
\"samples\": [\n   0.02\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe"}

```
pricing_missmatch_df = feature_sel_df[feature_sel_df['unit_price'] !=
feature_sel_df['list_price']]
list_price_to_unit_price = defaultdict(list)
for list_price in pricing_missmatch_df['list_price'].unique():
    list_price_to_unit_price[list_price] =
pricing_missmatch_df[pricing_missmatch_df['list_price'] == list_price]
['unit_price'].unique()
{len(unit_prices) for unit_prices in
list_price_to_unit_price.values()}
```

```
{1, 2, 3, 4, 5, 6, 8}
```

```
pricing_missmatch_df['product_id'] = merged_sales_df['product_id']
pricing_missmatch_df.groupby('product_id')[['list_price',
'unit_price']].nunique().head(10)
```

{"summary":"{\n \"name\": \"pricing_missmatch_df\",\n \"rows\": 10,\
n \"fields\": [\n   {\n   \"column\": \"product_id\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 10,\n   \"samples\": [\n
\"06bfa0ff-a56c-43d4-9226-f641c27a09ed\",\n   \"0025c2c9-855d-
45a7-92ad-f4ab2fb016d1\",\n   \"032eeb77-32d2-4ade-a122-
fd5873189326\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"list_price\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0,\n   \"min\": 1,\n
\"max\": 1,\n   \"num_unique_values\": 1,\n   \"samples\":
[\n   1\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"unit_price\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 1,\n   \"min\": 1,\n
\"max\": 7,\n   \"num_unique_values\": 5,\n   \"samples\":
[\n   2\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

The 'list_price' represents the standard or advertised price at which a product is intended to be sold. However, the 'unit_price' reflects the actual price at which the product was sold. This discrepancy can arise due to various factors. For a single product, the observation of varying 'unit_price' values against a consistent 'list_price' suggests different pricing strategies or discount applications. These variations are crucial to retain, as they can reveal patterns indicative of sales performance, promotional effectiveness, or potential pricing anomalies. Keeping both features allows for a comprehensive analysis of these pricing dynamics, which can be valuable for understanding customer behavior and optimizing future sales.

```
duplicate_features = [
    'discount_pct'
]

feature_sel_df.drop(duplicate_features, axis=1, inplace=True)

# <Student to fill this section>
feature_selection_5_insights = """
The 'discount_pct' feature was removed due to its redundancy with
'unit_price_discount' and other descriptive features. While
'discount_pct' indicates a
potential discount based on promotions or volume (e.g., 'Volume
Discount 11 to 14', 'Touring-1000 Promotion'), the
'unit_price_discount' accurately
reflects the actual discount applied to each sales order detail. In
cases where 'order_quantity' is 1, 'discount_pct' might show a non-
zero value, but the
actual 'unit_price_discount' would be zero, as the promotional
conditions were not met. Therefore, retaining 'discount_pct' would
introduce unnecessary
multicollinearity without adding unique signal, as the effective
discount is already captured by 'unit_price_discount'.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_5_insights',
value=feature_selection_5_insights)
```

```
<IPython.core.display.HTML object>
```

A.6 Approach 6 "Drop irrelevant discount metadata features"

```
display(feature_sel_df['type'].value_counts())
display(feature_sel_df['category'].value_counts())
```

```
type
No Discount              40081
Volume Discount           1621
New Product                327
Discontinued Product        33
Seasonal Discount           32
Excess Inventory             6
Name: count, dtype: int64

category
No Discount    40081
Reseller        2019
Name: count, dtype: int64
```

```
feature_sel_df.groupby('category')
['unit_price_discount'].value_counts()
```

```
category     unit_price_discount
No Discount  0.00                   40081
Reseller     0.00                    1172
             0.02                     309
             0.15                     233
             0.05                     148
             0.20                      98
             0.40                      33
             0.10                      20
             0.30                       6
Name: count, dtype: int64
```

```
feature_sel_df.groupby('max_quantity')
['order_quantity'].value_counts()
```

```
max_quantity  order_quantity
14.0          1.0               1156
              12.0               108
              11.0                98
              13.0                60
              14.0                43
24.0          16.0                38
              18.0                27
              15.0                26
              17.0                18
              19.0                14
              20.0                 9
              21.0                 6
              22.0                 4
              23.0                 4
              24.0                 2
40.0          25.0                 2
              26.0                 2
              27.0                 1
              28.0                 1
              29.0                 1
              36.0                 1
Name: count, dtype: int64
```

```
discount_metadat_cols = [
    'description',
    'category',
    'max_quantity'
]

feature_sel_df.drop(discount_metadat_cols, axis=1, inplace=True)

# <Student to fill this section>
feature_selection_6_insights = """
The 'description' feature, while offering textual details about a
```

```
specific offer or product, was removed due to its product-specific
nature. For anomaly
detection, broad, generalizable patterns are sought rather than
insights tied to individual product descriptions, which do not
contribute to identifying
unusual sales trends across a diverse product range. Similarly, the
'category' feature was dropped because it provided redundant
information already
encapsulated within the 'unit_price_discount'. Additionally, the
'max_quantity' feature is removed because, upon inspection, the
'order_quantity' never
exceeded the 'max_quantity', rendering 'max_quantity' an uninformative
boundary condition for sales anomaly detection.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_6_insights',
value=feature_selection_6_insights)
```

```
<IPython.core.display.HTML object>
```

A.7 Approach 7 "Drop irrelevant product metadata features"

```
product_metadata_cols = [
    'is_manufactured',
    'safety_stock_level',
    'reorder_point',
    'days_to_manufacture',
    'product_number',
    'standard_cost',
    'sell_start_date',
    'sell_end_date',
    'name',
    'color',
    'size',
    'size_unit_measure_code',
    'weight',
    'weight_unit_measure_code'
]

feature_sel_df.drop(product_metadata_cols, axis=1, inplace=True)

# <Student to fill this section>
feature_selection_7_insights = """
The features 'is_manufactured', 'safety_stock_level', 'reorder_point',
'days_to_manufacture', 'product_number', 'standard_cost',
'sell_start_date' and
'sell_end_date' are primarily related to product manufacturing,
internal inventory management, and logistical aspects. While important
for business
```

```
operations, these attributes do not directly influence or provide
signals for anomalous sales patterns from a customer transaction
perspective. The 'name'
feature, while descriptive, would introduce high cardinality, making
it less effective for direct modeling. Instead, using the 'category'
and 'subcategory'
of the product provides a more generalized and aggregated signal
without the issues of high cardinality. 'color' is also dropped
because for the purpose of
identifying sales anomalies, the specific color of a product is not
expected to be a primary driver or indicator of an unusual sales
event. While color
might be relevant for understanding customer preferences or predicting
sales volume, it is not directly linked to detecting deviations from
normal sales
behavior. 'size', 'size_unit_measure_code', 'weight', and
'weight_unit_measure_code' are also dropped as, from the perspective
of a sale, the only way
these factors can affect a sale price is through shipping price, but
we already have a shipping price feature - 'freight'. Similar to
'color', our focus is
related to sales anomaly detection, not trying to understand customer
behavior or product sales overall. Including these features would add
noise and
complexity to the model. Therefore, removing them helps to streamline
the feature set, ensuring that the model focuses on attributes
directly relevant to
observed sales behavior.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_7_insights',
value=feature_selection_7_insights)
```

```
<IPython.core.display.HTML object>
```

A.8 "Drop irrelevant sale features"

```
irrelevant_sale_cols = [
    'tax_amount',
    'freight',
    'due_date',
    'ship_date'
]

feature_sel_df.drop(irrelevant_sale_cols, axis=1, inplace=True)

# <Student to fill this section>
feature_selection_8_insights = """
The 'tax_amount' and 'freight' features represent values for the
```

```
complete sales order, not for individual products within that order.
Similarly, 'due_date'
and 'ship_date' provide information for the entire sales order. Our
business objective is to identify anomalous sales patterns at the
product level. Including
these aggregated sales-level features could introduce noise and mask
product-specific anomalies, as their values do not directly reflect
the characteristics
of a single product's sale. Therefore, these features are removed to
ensure the model focuses on attributes relevant to individual product
transactions,
aligning with the goal of detecting product-level anomalies.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_8_insights',
value=feature_selection_8_insights)
```

```
<IPython.core.display.HTML object>
```

A.9 Final Selection of Features

```
features_list = list(feature_sel_df.columns)

feature_selection_explanations = """
All other features not explicitly dropped were included, as they are
believed to provide meaningful signals for identifying anomalous sales
patterns.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_explanations',
value=feature_selection_explanations)
```

```
<IPython.core.display.HTML object>
```

B. Data Cleaning

```
df_clean = feature_sel_df.copy()
```

B.1 Fixing "Missing values for `product_line` feature"

```
df_clean[df_clean['product_line'].isna()].groupby('name_category')
['name_subcategory'].value_counts()
```

```
name_category  name_subcategory
Components     Derailleurs         147
               Brakes              133
```

```
                Cranksets           132
                Bottom Brackets     123
                Chains               83
                Headsets             26
                Forks                21
Name: count, dtype: int64
```

```
bool(df_clean[df_clean['name_subcategory'].isin(
    df_clean[df_clean['product_line'].isna()]
['name_subcategory'].unique())]['product_line'].isna().sum() ==
df_clean['product_line'].isna().sum())
```

```
True
```

```
df_clean.loc[df_clean['product_line'].isna(), 'product_line'] = 'O'
```

```
data_cleaning_1_explanations = """
During EDA of 'product' table, it was identified that the
'product_line' feature broadly classifies products into different bike
types. All products with a
NaN value in the 'product_line' column correspond to items not
specific to any particular bike type, indicating they are generic
components. All
subcategories that have missing 'product_line' value, only ever have
missing 'product_line' values within them. This indicates that the
missingness is
consistent within these subcategories. To address these missing values
and maintain data integrity, 'O' (representing "Others") has been
assigned to the
'product_line' for these products. This imputation ensures that all
products are categorized, allowing for more comprehensive analysis
without introducing
bias or losing valuable information about generic components.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_1_explanations',
value=data_cleaning_1_explanations)
```

```
<IPython.core.display.HTML object>
```

B.2 Fixing "Missing values for `class` feature"

```
df_clean[df_clean['class'].isna()].groupby('name_category')
['name_subcategory'].value_counts()
```

```
name_category  name_subcategory
Accessories    Tires and Tubes      5626
               Bottles and Cages    4082
               Helmets              3897
               Fenders               732
```

```
                Cleaners               534
                Hydration Packs        454
                Bike Racks             270
                Bike Stands            132
                Pumps                   33
                Locks                   30
Clothing        Jerseys               2538
                Caps                  1457
                Gloves                1071
                Shorts                 570
                Vests                  508
                Socks                  329
                Tights                 117
                Bib-Shorts              95
Components      Derailleurs            147
                Brakes                 133
                Chains                  83
                Pedals                  30
Name: count, dtype: int64
```

```
bool(df_clean[df_clean['name_subcategory'].isin(
    df_clean[df_clean['class'].isna()]['name_subcategory'].unique())]
['class'].isna().sum() == df_clean['class'].isna().sum())
```

```
True
```

```
df_clean.loc[df_clean['class'].isna(), 'class'] = 'O'
```

```
data_cleaning_2_explanations = """
During EDA of 'product' table, it was identified that the 'class'
feature broadly classifies products into different categories (H, M,
L). All products
with a NaN value in the 'class' column correspond to items not
specific to any particular classification. All subcategories that have
missing 'class'
value, only ever have missing 'class' values within them. This
indicates that the missingness is consistent within these
subcategories. To address these
missing values and maintain data integrity, 'O' (representing
"Others") has been assigned to the 'class' for these products. This
imputation ensures that
all products are categorized, allowing for more comprehensive analysis
without introducing bias or losing valuable information about generi
components.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_2_explanations',
value=data_cleaning_2_explanations)
```

```
<IPython.core.display.HTML object>
```

B.3 Fixing "Missing values for `style` feature"

```
df_clean[df_clean['style'].isna()].groupby('name_category')
['name_subcategory'].value_counts()
```

```
name_category  name_subcategory
Accessories    Tires and Tubes      8593
               Bottles and Cages    4082
               Helmets              3897
               Fenders               732
               Cleaners              534
               Hydration Packs       454
               Bike Racks            270
               Bike Stands           132
               Pumps                  33
               Locks                  30
Components     Pedals                283
               Handlebars            276
               Saddles               225
               Wheels                199
               Derailleurs           147
               Brakes                133
               Cranksets             132
               Bottom Brackets       123
               Chains                 83
               Headsets               26
               Forks                  21
Name: count, dtype: int64
```

```
bool(df_clean[df_clean['name_subcategory'].isin(
    df_clean[df_clean['style'].isna()]['name_subcategory'].unique())]
['style'].isna().sum() == df_clean['style'].isna().sum())
```

```
True
```

```
df_clean.loc[df_clean['style'].isna(), 'style'] = 'G'
```

```
data_cleaning_3_explanations = """
During EDA of 'product' table, it was identified that the 'style'
feature broadly classifies products based on their relevance to
gender. All products with a
NaN value in the 'style' column correspond to items not specific to
any particular style classification, indicating they are generic
components. All
subcategories that have missing 'style' value, only ever have missing
'style' values within them. This indicates that the missingness is
consistent within
these subcategories. To address these missing values and maintain data
integrity, 'G' (representing "Generic") has been assigned to the
'style' for these
products. This imputation ensures that all products are categorized,
```

```
allowing for more comprehensive analysis without introducing bias or
losing valuable
information about generic components.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_3_explanations',
value=data_cleaning_3_explanations)
```

```
<IPython.core.display.HTML object>
```

C. Split Datasets

```
training_df = df_clean.copy()
validation_df = None
testing_df = None

data_splitting_explanations = """
For unsupervised anomaly detection models, traditional data splitting
into training, validation and testing sets, as used in supervised
learning, is not
directly applicable. These models do not rely on a labelled target
variable to learn patterns or measure prediction accuracy. Instead,
they identify
observations that deviate from the majority of the data based on the
structure of the available features. In this project, the objective is
not to perform
novel semi-supervised training or to build a labelled classifier.
Therefore, no artificial target labels are created and no supervised
train-test evaluation
is performed.

The model is fitted on the available transaction data, which is
assumed to mostly represent normal business behaviour, and it learns
the local transaction
patterns from this data. Records that are sufficiently different from
their local neighbourhood are then flagged as potential anomalies.
Since there is no
ground-truth anomaly label, there is no explicit testing set in the
supervised sense to calculate accuracy, precision, recall or F1-score.
Instead, the
usefulness of the model is assessed through unsupervised and business-
focused indicators. Therefore, the focus is on building a robust
representation of
normal transaction behaviour and producing an actionable anomaly
shortlist rather than on supervised predictive validation.
"""
```

```
# Do not modify this code
print_tile(size="h3", key='data_splitting_explanations',
value=data_splitting_explanations)
```

```
<IPython.core.display.HTML object>
```

D. Feature Engineering

```
# DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets

try:
  training_df_eng = training_df.copy()
  validation_df_eng = validation_df.copy()
  testing_df_eng = testing_df.copy()
except Exception as e:
  print(e)
```

```
'NoneType' object has no attribute 'copy'
```

```
# Error is observed because validationa and testing dataframe is None
as no splitting is carried out.
```

D.1 New Feature "Offer Duration"

```
training_df_eng['order_date'] =
pd.to_datetime(training_df_eng['order_date'])
training_df_eng['start_date'] =
pd.to_datetime(training_df_eng['start_date'])

training_df_eng['offer_duration_days'] =
(training_df_eng['order_date'] -
training_df_eng['start_date']).dt.days
display(training_df_eng[['order_date', 'start_date',
'offer_duration_days']].sample(10))
display(training_df_eng[['offer_duration_days']].describe(include='all
'))
```

```
{"summary":"{\n  \"name\":
\"display(training_df_eng[['offer_duration_days']]\",\n  \"rows\":
10,\n  \"fields\": [\n    {\n      \"column\": \"order_date\",\n
\"properties\": {\n        \"dtype\": \"date\",\n        \"min\":
\"2011-07-23 22:00:00\",\n        \"max\": \"2014-04-27 22:00:00\",\n
\"num_unique_values\": 10,\n        \"samples\": [\n          \"2012-
06-29 22:00:00\",\n          \"2014-01-05 23:00:00\",\n
\"2013-06-02 22:00:00\"\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"start_date\",\n      \"properties\": {\n
```

\"dtype\": \"date\",\n   \"min\": \"2011-05-01 00:00:00\",\n
\"max\": \"2011-05-01 00:00:00\",\n   \"num_unique_values\": 1,\n
\"samples\": [\n   \"2011-05-01 00:00:00\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"offer_duration_days\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
298,\n   \"min\": 83,\n   \"max\": 1092,\n
\"num_unique_values\": 10,\n   \"samples\": [\n   425\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   }\n]\n}","type":"dataframe"}

{"summary":"{\n \"name\":
\"display(training_df_eng[['offer_duration_days']]\",\n \"rows\": 8,\
n \"fields\": [\n   {\n   \"column\": \"offer_duration_days\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
14635.529176189028,\n   \"min\": -1.0,\n   \"max\":
42100.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n
885.5765795724466,\n   929.0,\n   42100.0\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe"}

```
training_df_eng.drop(['start_date'], axis=1, inplace=True)

training_df_eng[training_df_eng['offer_duration_days'] < 0][['type',
'online_order_flag', 'offer_duration_days']].value_counts()
```

```
type                 online_order_flag  offer_duration_days
New Product          0.0                -1                    59
Seasonal Discount    0.0                -1                    32
Discontinued Product 0.0                -1                    17
Excess Inventory     0.0                -1                     2
Name: count, dtype: int64
```

```
feature_engineering_1_explanations = """
The 'order_date' and 'start_date' features, initially strings, have
been converted into datetime objects. A new numerical feature,
'offer_duration_days',
has been created by calculating the difference in days between the
'order_date' and the 'start_date' of the special offer. This feature
is crucial for
anomaly detection models as it quantifies how long after an offer
began a product was purchased, providing a direct measure of the
offer's impact over time.
This allows the model to capture patterns related to offer timeliness
or unusual purchasing delays, helping to identify anomalies that might
be related to
products being sold significantly earlier or later than expected
relative to the offer period. This numerical representation offers a
continuous scale for
the model to detect deviations in purchasing patterns during offers.
```

```
The raw date columns were also dropped as the new feature encapsulates
the relevant
temporal information. Negative values in 'offer_duration_days'
indicate that the product was ordered before the special offer began.
This scenario could
represent anomalies such as pre-orders or data entry errors, or it
might highlight specific operational patterns where customers secure
products prior to
official sales.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_1_explanations',
value=feature_engineering_1_explanations)
```

```
<IPython.core.display.HTML object>
```

D.2 New Feature "Offer End Duration"

```
training_df_eng['end_date'] =
pd.to_datetime(training_df_eng['end_date'])

training_df_eng['offer_end_duration_days'] =
(training_df_eng['end_date'] - training_df_eng['order_date']).dt.days
display(training_df_eng[['order_date', 'end_date',
'offer_end_duration_days']].sample(10))
display(training_df_eng[['offer_end_duration_days']].describe(include=
'all'))
```


```
{"summary":"{\n  \"name\":
\"display(training_df_eng[['offer_end_duration_days']]\",\n  \"rows\":
10,\n  \"fields\": [\n    {\n      \"column\": \"order_date\",\n
\"properties\": {\n        \"dtype\": \"date\",\n        \"min\":
\"2012-12-30 23:00:00\",\n        \"max\": \"2014-06-29 22:00:00\",\n
\"num_unique_values\": 10,\n        \"samples\": [\n          \"2014-
06-29 22:00:00\",\n          \"2013-03-29 23:00:00\",\n
\"2013-11-28 23:00:00\"\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"end_date\",\n      \"properties\": {\n        \"dtype\":
\"date\",\n        \"min\": \"2014-05-30 00:00:00\",\n        \"max\":
\"2014-11-30 00:00:00\",\n        \"num_unique_values\": 2,\n
\"samples\": [\n          \"2014-05-30 00:00:00\",\n          \"2014-
11-30 00:00:00\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"offer_end_duration_days\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 182,\n        \"min\": 153,\n
\"max\": 699,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          153,\n          610\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```

{"summary":"{\n \"name\":
\"display(training_df_eng[['offer_end_duration_days']]\",\n \"rows\":
8,\n \"fields\": [\n   {\n   \"column\":
\"offer_end_duration_days\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 14738.036354333324,\n
\"min\": 0.0,\n   \"max\": 42100.0,\n
\"num_unique_values\": 8,\n   \"samples\": [\n
402.5941330166271,\n   366.0,\n   42100.0\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe"}

```
training_df_eng.drop(['order_date', 'end_date'], axis=1, inplace=True)
```

```
feature_engineering_2_explanations = """
The 'end_date' feature, initially a string, has been converted into a
datetime object. A new numerical feature, 'offer_end_duration_days',
has been created
by calculating the difference in days between the 'end_date' of the
special offer and the 'order_date'. This feature provides another
temporal dimension
for anomaly detection, indicating how long before or after an offer's
conclusion a product was purchased. Positive values in
'offer_end_duration_days'
indicate that the product was ordered before the offer ended, which is
expected for valid purchases. Zero values indicate a last minute
purchase before the
end of offer. This could highlight anomalies such as delayed
processing of orders, or system errors. Such deviations from expected
behavior are crucial for
identifying unusual sales patterns. The original 'order_date' and
'end_date' columns were subsequently dropped as the new
'offer_end_duration_days' feature
encapsulates the relevant temporal information more effectively for
anomaly detection.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_2_explanations',
value=feature_engineering_2_explanations)
```

```
<IPython.core.display.HTML object>
```

```
training_df_eng.columns
```

```
Index(['order_quantity', 'unit_price', 'unit_price_discount',
       'revision_number', 'online_order_flag', 'min_quantity', 'type',
       'list_price', 'product_line', 'class', 'style',
'name_subcategory',
       'name_category', 'offer_duration_days',
'offer_end_duration_days'],
      dtype='object')
```

E. Data Preparation for Modeling

```
# DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets

try:
  X_train = training_df_eng.copy()
  X_val = validation_df_eng.copy()
  X_test = testing_df_eng.copy()
except Exception as e:
  print(e)
```

```
name 'validation_df_eng' is not defined
```

```
# Error is observed because validationa and testing dataframe is None
as no splitting is carried out.
```

E.1 Data Transformation "Scale numerical features"

```
numerical_features = X_train.select_dtypes(include=['int64',
'float64']).columns.tolist()
numerical_features
```

```
['order_quantity',
 'unit_price',
 'unit_price_discount',
 'revision_number',
 'online_order_flag',
 'min_quantity',
 'list_price',
 'offer_duration_days',
 'offer_end_duration_days']
```

```
scaler = StandardScaler()
X_train[numerical_features] =
scaler.fit_transform(X_train[numerical_features])

pickle.dump(scaler, open(at.folder_path / 'scaler.pkl', 'wb'))

# <Student to fill this section and then remove this comment>
data_transformation_1_explanations = """
The StandardScaler is applied to numerical features. This is a crucial
preprocessing step for bringing all features to comparable values.
This also ensures
that each feature contributes equally to the distance metrics and the
model learns unbiased patterns. The scaler is also saved so that
later, after an
anomaly is detected, the values can be reverted to their normal state
for better interpretability and to understand the reason for the
```

```
anomaly.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_1_explanations',
value=data_transformation_1_explanations)
```

```
<IPython.core.display.HTML object>
```

E.2 Data Transformation "Encode categorical features"

```
categorical_features =
X_train.select_dtypes(include=['object']).columns.tolist()
categorical_features
```

```
['type', 'product_line', 'class', 'style', 'name_subcategory',
'name_category']
```

```
X_train = pd.get_dummies(X_train, columns=categorical_features)

data_transformation_2_explanations = """
The categorical features ('product_line', 'class', 'style',
'name_subcategory', 'name_category') are transformed using one-hot
encoding. This data
transformation is essential for as this converts these non-numeric
categories into a numerical format that models can understand and
utilize. Assigning
arbitrary numerical labels (e.g., 0, 1, 2) to categorical features can
implicitly introduce an artificial ordinal relationship that doesn't
exist in the
data. One-hot encoding creates a new binary column for each category,
where a '1' indicates the presence of that category and a '0'
indicates its absence.
This avoids imposing any false ordinal relationships.
"""

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_2_explanations',
value=data_transformation_2_explanations)
```

```
<IPython.core.display.HTML object>
```

F. Save Datasets

Do not change this code

```
# DO NOT MODIFY THE CODE IN THIS CELL

try:
```

```
  X_train.to_csv(at.folder_path / 'X_train.csv', index=False)

  X_val.to_csv(at.folder_path / 'X_val.csv', index=False)

  X_test.to_csv(at.folder_path / 'X_test.csv', index=False)
except Exception as e:
  print(e)
```

```
name 'X_val' is not defined
```

```
# Error is observed because validationa and testing dataframe is None
as no splitting is carried out.
```